COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structures and Algorithms | **SECTION:** | CPE21S3 |
| **DATE PERFORMED:** | 07 / 16 / 2026 | **DATE SUBMITTED:** | |
| **NAME:** | Elijah I. Dizor | **INSTRUCTOR:** | Engr. Jimlord Quejado |

## ACTIVITY NO. 2.1 [Arrays, Pointers and Dynamic Memory Allocation]

### 1. PROCEDURE OUTPUT

What procedures did you perform?

### 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers

| Jenna's Grocery List | | |
|---|---|---|
| Apple | PHP 10 | x7 |
| Banana | PHP 10 | x8 |
| Broccoli | PHP 60 | x12 |
| Lettuce | PHP 50 | x10 |

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

COMPUTER ENGINEERING

QUEZON CITY

SCREENSHOT

```cpp
#include <iostream>
#include <string>

class Fruit {
private:
    std::string name;
    double price;
    int quantity;

public:
    Fruit(std::string n = "", double p = 0.0, int q = 0) : name(n), price(p), quantity(q) {}

    ~Fruit() {}

    Fruit(const Fruit& other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
    }

    Fruit& operator=(const Fruit& other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
        }
        return *this;
    }

    double calculateSum() const {
        return price * quantity;
    }

    void printDetails() const {
        std::cout << name << " | Price: Php " << price << " | Qty: " << quantity << " | Total: Php " << calculateSum() << std::endl;
    }
};

class Vegetable {
private:
    std::string name;
    double price;
    int quantity;

public:
    Vegetable(std::string n = "", double p = 0.0, int q = 0) : name(n), price(p), quantity(q) {}

    ~Vegetable() {}

    Vegetable(const Vegetable& other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
    }

    Vegetable& operator=(const Vegetable& other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
        }
        return *this;
    }

    double calculateSum() const {
        return price * quantity;
    }

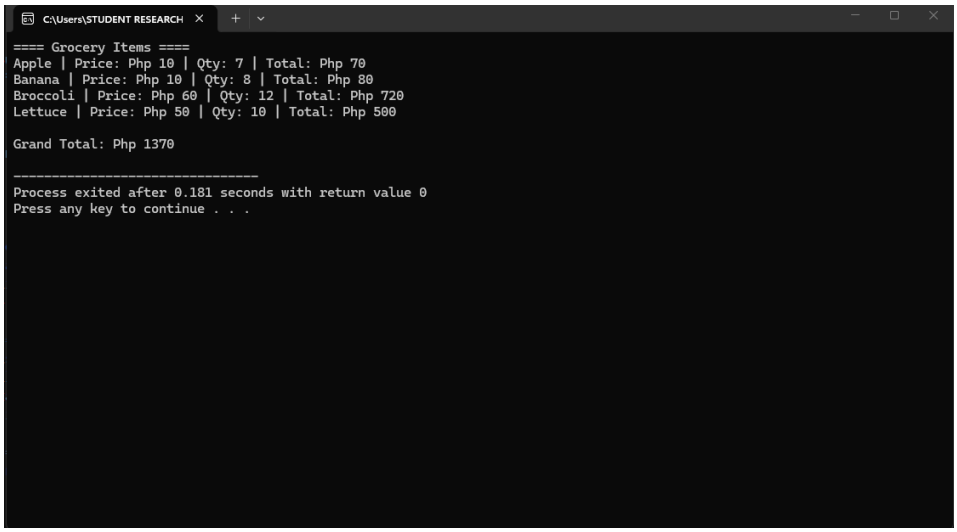
    void printDetails() const {
        std::cout << name << " | Price: Php " << price << " | Qty: " << quantity << " | Total: Php " << calculateSum() << std::endl;
    }
};

int main() {
    Fruit apple("Apple", 10.0, 7);
    Fruit banana("Banana", 10.0, 8);
    Vegetable broccoli("Broccoli", 60.0, 12);
    Vegetable lettuce("Lettuce", 50.0, 10);

    std::cout << "=== Grocery Items ===" << std::endl;
    apple.printDetails();
    banana.printDetails();
    broccoli.printDetails();
    lettuce.printDetails();

    double totalCost = apple.calculateSum() + banana.calculateSum() + broccoli.calculateSum() + lettuce.calculateSum();
    std::cout << "\nGrand Total: Php " << totalCost << std::endl;

    return 0;
}
```

| | |
|---|---|
| OUTPUT |  |
| OBSERVATION | Fruit and vegetable classes had identical data fields, logic, formula, and methods. Both are unrelated types in the compiler and cannot be grouped together into a single array or collection. As it forces it to manually call printDetails() and add up calculateSum() individually on each item |



Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List.

| | |
|---|---|
| SCREENSHOT | |

```cpp
#include <iostream>
#include <string>

class GroceryItem {
protected:
    std::string name;
    double price;
    int quantity;
    std::string type;

public:
    GroceryItem(std::string n = "", double p = 0.0, int q = 0, std::string t = "")
        : name(n), price(p), quantity(q), type(t) {}

    virtual ~GroceryItem() {}

    GroceryItem(const GroceryItem& other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
        type = other.type;
    }

    GroceryItem& operator=(const GroceryItem& other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
            type = other.type;
        }
        return *this;
```

```
    }

    double calculateSum() const {
        return price * quantity;
    }

    void printDetails() const {
        std::cout << "[" << type << "] " << name
                  << " | Price: PHP " << price
                  << " | Qty: " << quantity
                  << " | Total: PHP " << calculateSum() << std::endl;
    }
};

class Fruit : public GroceryItem {
public:
    Fruit(std::string n = "", double p = 0.0, int q = 0)
        : GroceryItem(n, p, q, "Fruit") {}
};

class Vegetable : public GroceryItem {
public:
    Vegetable(std::string n = "", double p = 0.0, int q = 0)
        : GroceryItem(n, p, q, "Vegetable") {}
};
```

```
int main() {
    const int totalItems = 4;
    GroceryItem* GroceryList[totalItems];

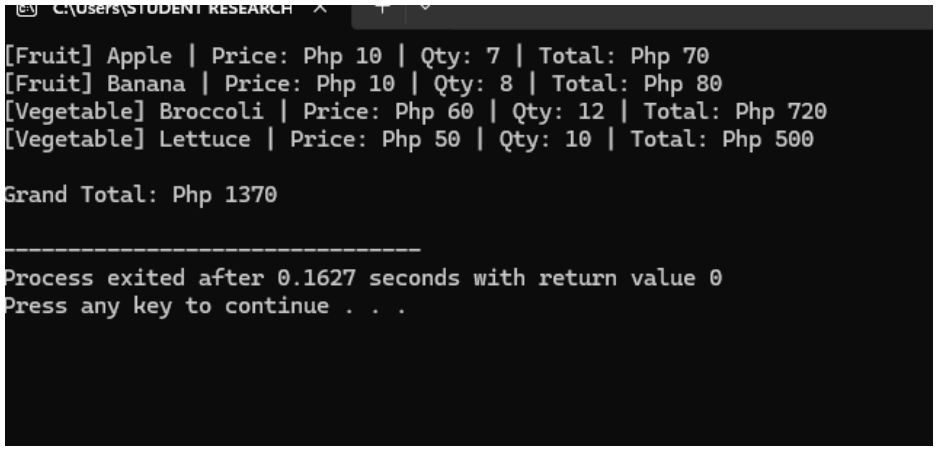
    GroceryList[0] = new Fruit("Apple", 10.0, 7);
    GroceryList[1] = new Fruit("Banana", 10.0, 8);
    GroceryList[2] = new Vegetable("Broccoli", 60.0, 12);
    GroceryList[3] = new Vegetable("Lettuce", 50.0, 10);

    double grandTotal = 0.0;
    for (int i = 0; i < totalItems; i++) {
        GroceryList[i]->printDetails();
        grandTotal += GroceryList[i]->calculateSum();
    }

    std::cout << "\nGrand Total: PHP " << grandTotal << std::endl;

    for (int i = 0; i < totalItems; i++) {
        delete GroceryList[i];
    }

    return 0;
}
```

| | |
|---|---|
| OUTPUT |  |
| OBSERVATION | The GroceryItem base class, it stores different derived objects inside a single pointer array (GroceryList), as it process every item using a single, unified for loop statement |



You must then access each saved instance and display all details about the items.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

SCREENSHOT

```cpp
#include <iostream>
#include <string>

class GroceryItem {
protected:
    std::string name;
    double price;
    int quantity;
    std::string type;

public:
    GroceryItem(std::string n = "", double p = 0.0, int q = 0, std::string t = "")
        : name(n), price(p), quantity(q), type(t) {}

    virtual ~GroceryItem() {}

    GroceryItem(const GroceryItem& other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
        type = other.type;
    }

    GroceryItem& operator=(const GroceryItem& other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
            type = other.type;
        }
        return *this;
```

```
    }

    double calculateSum() const {
        return price * quantity;
    }

    void printDetails() const {
        std::cout << "[" << type << "] " << name
                  << " | Price: PHP " << price
                  << " | Qty: " << quantity
                  << " | Total: PHP " << calculateSum() << std::endl;
    }
};

class Fruit : public GroceryItem {
public:
    Fruit(std::string n = "", double p = 0.0, int q = 0)
        : GroceryItem(n, p, q, "Fruit") {}
};

class Vegetable : public GroceryItem {
public:
    Vegetable(std::string n = "", double p = 0.0, int q = 0)
        : GroceryItem(n, p, q, "Vegetable") {}
};
```

```
double TotalSum(GroceryItem* list[], int size) {
    double total = 0.0;
    for (int i = 0; i < size; i++) {
        total += list[i]->calculateSum();
    }
    return total;
}

int main() {
    const int totalItems = 4;
    GroceryItem* GroceryList[totalItems];

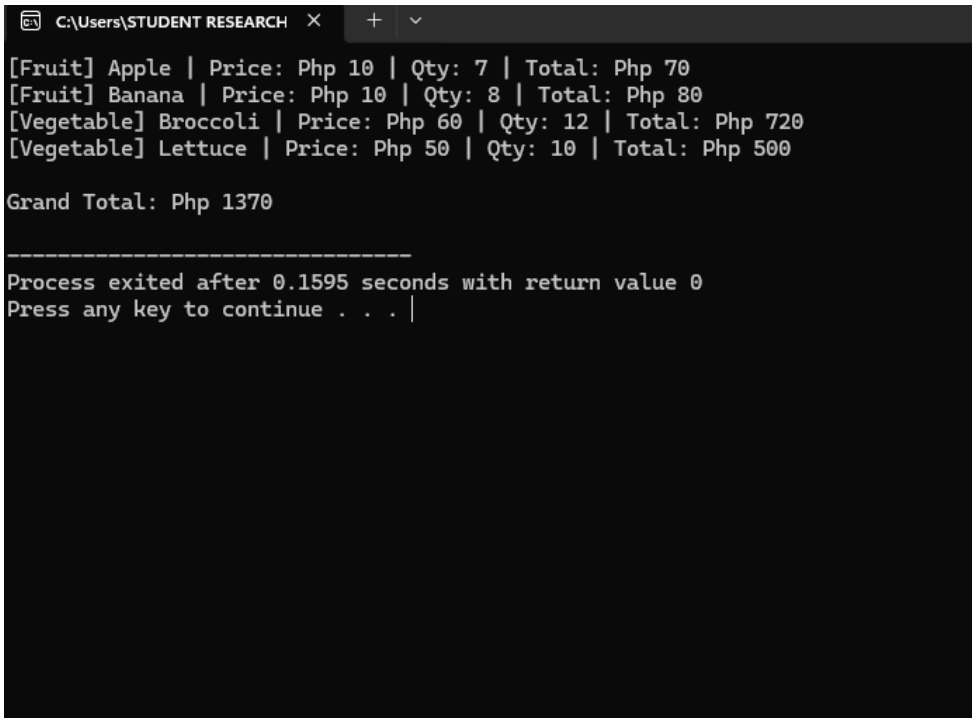
    GroceryList[0] = new Fruit("Apple", 10.0, 7);
    GroceryList[1] = new Fruit("Banana", 10.0, 8);
    GroceryList[2] = new Vegetable("Broccoli", 60.0, 12);
    GroceryList[3] = new Vegetable("Lettuce", 50.0, 10);

    for (int i = 0; i < totalItems; i++) {
        GroceryList[i]->printDetails();
    }

    double grandTotal = TotalSum(GroceryList, totalItems);
    std::cout << "\nGrand Total: PHP " << grandTotal << std::endl;

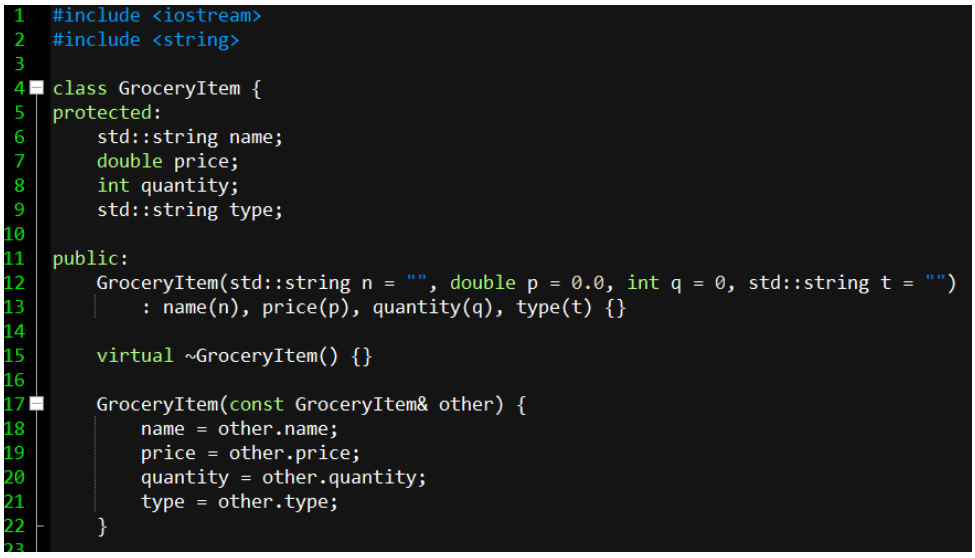
    for (int i = 0; i < totalItems; i++) {
        delete GroceryList[i];
    }

    return 0;
}
```

| | |
|---|---|
| OUTPUT |  |
| OBSERVATION | Totalsum cleans up the main execution path and acts as an orchestrator for setup, display, and breakdown. The manual copy constructor and copy assignment operator implementations remain inside Groceryitem. |



Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

| | |
|---|---|
| SCREENSHOT |  |

```cpp
#include <iostream>
#include <string>

class GroceryItem {
protected:
    std::string name;
    double price;
    int quantity;
    std::string type;

public:
    GroceryItem(std::string n = "", double p = 0.0, int q = 0, std::string t = "")
        : name(n), price(p), quantity(q), type(t) {}

    virtual ~GroceryItem() {}

    GroceryItem(const GroceryItem& other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
        type = other.type;
    }
```

```
    GroceryItem& operator=(const GroceryItem& other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
            type = other.type;
        }
        return *this;
    }

    double calculateSum() const {
        return price * quantity;
    }

    void printDetails() const {
        std::cout << "[" << type << "] " << name
                  << " | Price: Php " << price
                  << " | Qty: " << quantity
                  << " | Total: Php " << calculateSum() << std::endl;
    }
};
```

```
class Fruit : public GroceryItem {
public:
    Fruit(std::string n = "", double p = 0.0, int q = 0)
        : GroceryItem(n, p, q, "Fruit") {}
};

class Vegetable : public GroceryItem {
public:
    Vegetable(std::string n = "", double p = 0.0, int q = 0)
        : GroceryItem(n, p, q, "Vegetable") {}
};

double TotalSum(GroceryItem* list[], int size) {
    double total = 0.0;
    for (int i = 0; i < size; i++) {
        if (list[i] != nullptr) {
            total += list[i]->calculateSum();
        }
    }
    return total;
}

```

```cpp
int main() {
    int totalItems = 4;
    GroceryItem* GroceryList[4];

    GroceryList[0] = new Fruit("Apple", 10.0, 7);
    GroceryList[1] = new Fruit("Banana", 10.0, 8);
    GroceryList[2] = new Vegetable("Broccoli", 60.0, 12);
    GroceryList[3] = new Vegetable("Lettuce", 50.0, 10);

    std::cout << "===== Original List =====" << std::endl;
    for (int i = 0; i < totalItems; i++) {
        GroceryList[i]->printDetails();
    }
    std::cout << "Original Total: Php " << TotalSum(GroceryList, totalItems) << std::endl;

    delete GroceryList[3];
    GroceryList[3] = nullptr;
    totalItems--;

    std::cout << "\n--- Updated List ---" << std::endl;
    for (int i = 0; i < totalItems; i++) {
        GroceryList[i]->printDetails();
    }
    std::cout << "New Total: Php " << TotalSum(GroceryList, totalItems) << std::endl;

    for (int i = 0; i < totalItems; i++) {
        delete GroceryList[i];
    }

    return 0;
}
```

OUTPUT

```
===== Original List =====
[Fruit] Apple | Price: Php 10 | Qty: 7 | Total: Php 70
[Fruit] Banana | Price: Php 10 | Qty: 8 | Total: Php 80
[Vegetable] Broccoli | Price: Php 60 | Qty: 12 | Total: Php 720
[Vegetable] Lettuce | Price: Php 50 | Qty: 10 | Total: Php 500
Original Total: Php 1370

--- Updated List ---
[Fruit] Apple | Price: Php 10 | Qty: 7 | Total: Php 70
[Fruit] Banana | Price: Php 10 | Qty: 8 | Total: Php 80
[Vegetable] Broccoli | Price: Php 60 | Qty: 12 | Total: Php 720
New Total: Php 870

--------------------------------
Process exited after 0.164 seconds with return value 0
Press any key to continue . . .
```

OBSERVATION

The totalsum function now includes a list[i] ! = nullptr, to prevent null pointers dereferencing crashes when calculating totals on sparse arrays. When you execute delete GroceryList[3]; and decrement totalItems to 3, the code assumes the remaining items occupy the indices of 0,1, and 2.

COMPUTER ENGINEERING

QUEZON CITY

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **chatgpt**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Code assistance and explanation of concepts**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **I used ai to explain concepts that help me understand it more. Code debugging or use of other c++ compiler extensions due to an outdated version of dev c++**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

"I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own."